

Homework 7 (Practice / Ungraded)

Professor: Dr. Mudassir Shabbir

No submission required.

Topics. Network flows and approximation algorithms.

1. **Maximum Flow by Example.** Consider the flow network $G = (V, E, c)$ with $V = \{s, a, b, c, d, t\}$ and the following directed edges and capacities:

$$s \rightarrow a (10), \quad s \rightarrow c (10), \quad a \rightarrow b (4), \quad a \rightarrow d (8), \quad c \rightarrow d (9), \quad b \rightarrow t (10), \quad d \rightarrow b (6), \quad d \rightarrow t (10).$$

- Find a maximum s - t flow by tracing Ford-Fulkerson. Show each augmenting path and the resulting residual graph.
- Identify a minimum s - t cut $(S, V \setminus S)$ and verify that its capacity equals the max-flow value.

2. **Ford-Fulkerson: Termination and Running Time.**

- Prove that when all edge capacities are positive integers, Ford-Fulkerson terminates and correctly computes a maximum flow. Show that the number of augmenting-path iterations is $O(f^*)$, where f^* is the max-flow value.
- Construct an explicit graph (the “bad example”) with 4 vertices and integer capacities in which some sequence of augmenting-path choices forces $\Theta(f^*)$ iterations, showing the bound above is tight.
- Explain how replacing DFS/arbitrary search with BFS (Edmonds-Karp) improves the running time to $O(VE^2)$, independent of capacity values. (You need not re-derive the bound; state and briefly explain it.)

3. **2-Approximation for Vertex Cover.** In the **Minimum Vertex Cover** problem, given an undirected graph $G = (V, E)$, find a smallest set $C \subseteq V$ such that every edge has at least one endpoint in C . **The maximal-matching 2-approximation algorithm:** find any maximal matching M and include both endpoints of each matched edge.

- Prove the output is a valid vertex cover.
- Prove the approximation ratio: $|C_{\text{ALG}}| \leq 2 |C_{\text{OPT}}|$.
- Give a family of graphs on which the algorithm outputs a cover of size exactly $2 |C_{\text{OPT}}|$, showing the factor of 2 is tight.

4. **2-Approximation for Metric TSP.**

In the **Metric Traveling Salesman Problem (Metric TSP)**, we are given a complete undirected graph K_n with edge weights $d : V \times V \rightarrow \mathbb{R}_{\geq 0}$ satisfying the *triangle inequality*: $d(u, w) \leq d(u, v) + d(v, w)$ for all $u, v, w \in V$. The goal is to find a minimum-weight Hamiltonian cycle (tour).

- Describe the MST-based 2-approximation algorithm for Metric TSP.
- Prove the algorithm produces a valid tour.
- Prove the approximation ratio: $\text{cost}(\text{ALG}) \leq 2 \cdot \text{OPT}$. (*Hint:* Lower-bound OPT by the MST cost, then argue that the pre-order DFS walk can be shortcut using the triangle inequality.)
- Explain why the triangle inequality is crucial: show that for general TSP (no metric assumption), no polynomial-time ρ -approximation exists for any constant $\rho \geq 1$ unless $P = NP$. (*Hint:* Give a gap-preserving reduction from Hamiltonian Cycle.)
- (*Challenge*) Briefly describe the Christofides–Serdyukov algorithm and state why it achieves a $\frac{3}{2}$ -approximation. Explain why matching the odd-degree vertices in the MST reduces the factor from 2 to $\frac{3}{2}$.

5. 2-Approximation for Max Cut.

In the **Maximum Cut** problem, given an undirected graph $G = (V, E)$, find a partition $(S, V \setminus S)$ that maximizes the number of edges crossing the cut.

A simple greedy algorithm: process vertices one at a time and places each vertex on the side (S or $\bar{S}$) that maximizes the number of new crossing edges.

- Prove that the greedy algorithm always produces a cut with at least $|E|/2$ crossing edges.
- Conclude that this is a 2-approximation: $\text{ALG} \geq \text{OPT}/2$. (*Note:* Since $\text{OPT} \leq |E|$, the bound follows directly.)
- Show that a *random* partition—each vertex placed independently in S with probability $\frac{1}{2}$ —also yields an expected $|E|/2$ crossing edges. Is this also a 2-approximation? Justify your answer.
- (*Challenge*) Explain at a high level how the Goemans–Williamson semidefinite-programming rounding algorithm improves the ratio to approximately 0.878 (i.e., $\text{ALG} \geq 0.878 \cdot \text{OPT}$).

6. Makespan Minimization: List Scheduling.

In **Makespan Minimization** ($P||C_{\max}$), we assign n jobs with processing times $p_1, \dots, p_n \geq 0$ to m identical parallel machines. The *makespan* is $\max_i(\text{total load on machine } i)$; the goal is to minimize it.

The **List Scheduling** algorithm (Graham's LS rule): assign each job in arbitrary order to the currently least-loaded machine.

- Let $C_{\max}^{\text{LS}}$ be the List Scheduling makespan and $C_{\max}^*$ the optimum. Prove:

$$C_{\max}^{\text{LS}} \leq \left(2 - \frac{1}{m}\right) C_{\max}^*.$$

(*Hint:* Let j^* be the last job to finish. Lower-bound $C_{\max}^*$ by (i) p_{j^*} and (ii) $\frac{1}{m} \sum_i p_i$. Combine these to get the ratio.)

- Construct a family of instances (parameterized by m) showing the bound $2 - \frac{1}{m}$ is tight.

7. Makespan Minimization: Longest Processing Time. The LPT rule sorts jobs in non-increasing order of processing time ($p_1 \geq p_2 \geq \dots \geq p_n$) and then applies List Scheduling.

- Give an example where List Scheduling performs worse than LPT. Explain intuitively why sorting helps.
- State (without full proof) that LPT achieves a $(\frac{4}{3} - \frac{1}{3m})$ -approximation.
- Prove the $\frac{4}{3}$ bound for the special case $m = 2$: show that on two machines, LPT achieves a $\frac{4}{3}$ -approximation. (*Hint:* If there are at most 3 jobs the claim is immediate. Otherwise argue about the processing time of the last job to finish relative to $C_{\max}^*$, using the sorted order.)

8. Greedy Set Cover ($\ln n$ -Approximation). Recall that in **Set Cover**, given a universe U with $|U| = n$ and a collection $\mathcal{S} = \{S_1, \dots, S_m\}$ of subsets of U , we seek the fewest sets covering U .

The greedy algorithm: repeatedly add the set covering the most uncovered elements until U is covered.

- Prove that the greedy algorithm returns a cover of size at most $H_n \cdot |\text{OPT}|$, where $H_n = 1 + \frac{1}{2} + \dots + \frac{1}{n} = \ln n + O(1)$. (*Hint:* After OPT sets cover U , some set must cover at least $1/|\text{OPT}|$ of the remaining elements; use this to bound the potential function drop.)
- Exhibit an instance showing the $\ln n$ factor is essentially tight for the greedy algorithm.
- State (without proof) the inapproximability result: Set Cover cannot be approximated within $(1 - \varepsilon) \ln n$ for any $\varepsilon > 0$ unless $P = NP$. Conclude that the greedy algorithm is essentially optimal.

9. **2-Approximation for Bin Packing.** In the **Bin Packing** problem, given n items with sizes $s_1, \dots, s_n \in (0, 1]$ and bins of capacity 1, find the minimum number of bins needed to pack all items.

The **Next Fit (NF)** algorithm: maintain a single “current” bin; place item i into the current bin if it fits, otherwise close that bin, open a new one, and place item i there.

- Let $\text{NF}(I)$ denote the number of bins used by Next Fit on instance I , and let $S = \sum_{i=1}^n s_i$ be the total item size. Prove that $\text{NF}(I) \leq 2\lceil S \rceil \leq 2\text{OPT}(I)$. (*Hint:* Consider any two consecutive bins opened by NF. Argue that the combined size of items in those two bins exceeds 1, so $2S > \text{NF}(I) - 1$.)
- Conclude that Next Fit is a 2-approximation for Bin Packing.
- Show the factor of 2 is essentially tight: construct a family of instances on which $\text{NF}(I) = 2\text{OPT}(I) - 2$. (*Hint:* Alternate items of size $\frac{1}{2} + \varepsilon$ and $\frac{1}{2} - \varepsilon$ for small $\varepsilon > 0$.)
- (Optional) State without proof that **First Fit Decreasing (FFD)**—sort items in non-increasing order, then greedily place each item in the first bin it fits—achieves an asymptotic ratio of $\frac{11}{9}$.

10. **2-Approximation for Subset Sum.** In the optimization version of **Subset Sum**, given positive integers $a_1, \dots, a_n$ and a target T , find a subset $S \subseteq \{1, \dots, n\}$ that maximizes $\sum_{i \in S} a_i$ subject to $\sum_{i \in S} a_i \leq T$.

The greedy algorithm:

- Remove all items with $a_i > T$.
 - Sort the remaining items in non-increasing order: $a_1 \geq a_2 \geq \dots \geq a_n$.
 - Maintain a running sum $S \leftarrow 0$; for each i , if $S + a_i \leq T$ then set $S \leftarrow S + a_i$.
 - Return S .
- Prove that the greedy achieves $\text{ALG} \geq T/2$ whenever at least one item satisfies $a_i \leq T$. (*Hint:* Split into two cases: (i) the largest item $a_1 \geq T/2$ —the first item added immediately gives $\text{ALG} \geq T/2$; (ii) all items $< T/2$ —suppose for contradiction that $\text{ALG} < T/2$ and show that no item was ever skipped, so the greedy sum equals the sum of all items, which is at most T , giving $\text{ALG} = \text{OPT}$.)
 - Conclude this is a 2-approximation: $\text{ALG} \geq \text{OPT}/2$. (*Note:* $\text{OPT} \leq T$ always, so $T/2 \geq \text{OPT}/2$.)
 - Give an example showing the ratio can be close to 2.
 - Explain why the exact problem is NP-hard yet the greedy gives a clean polynomial-time 2-approximation. What structural property does the greedy exploit?

11. **Approximation for the Partition Problem.** In the optimization version of the **Partition Problem**, given positive integers $a_1, \dots, a_n$ with sum $W = \sum_{i=1}^n a_i$, find a partition into two subsets A and $B = \{1, \dots, n\} \setminus A$ that minimizes $\max(\sum_{i \in A} a_i, \sum_{i \in B} a_i)$.

- Observe that $\text{OPT} \geq W/2$ (the larger half is at least half the total) and $\text{OPT} \geq \max_i a_i$ (the item alone lower-bounds the max). Combine these into a single lower bound.
- Show that the Partition Problem is a special case of Makespan Minimization ($P \parallel C_{\max}$) with $m = 2$ machines.
- Applying the List Scheduling bound from Problem 7, conclude directly that the greedy “assign each item to the lighter pile” achieves a $(2 - \frac{1}{2}) = \frac{3}{2}$ -approximation for Partition.
- Using the LPT bound from Problem 8 with $m = 2$, show that sorting items in non-increasing order before the greedy assignment achieves a $\frac{4}{3}$ -approximation.
- Give a concrete instance with 4 items on which the unsorted greedy achieves makespan $\frac{3}{2} \text{OPT}$ but LPT achieves the optimum.